

Sara Manzoor

Deep Learning Revision Notes

Sequential Models for Deep Learning

AI/ML Engineering Revision Journey — Module 5 | Day 3

Sequential Data · RNN · LSTM · GRU · Bidirectional RNN ·
Sequence-to-Sequence (Seq2Seq)

Table of Contents

1. Sequential Models
 2. RNN (Recurrent Neural Network)
 3. LSTM (Long Short-Term Memory)
 4. GRU (Gated Recurrent Unit)
 5. Bidirectional RNN
 6. Sequence-to-Sequence Models (Seq2Seq)
- Key Takeaways

1. Sequential Models

What is Sequential Data?

Sequential Data is data in which order matters — data points are not independent of one another; instead, each point exists within a sequence where it depends on the points that came before it. If the order of a sequence is shuffled, its meaning is lost or changed entirely.

Example: "I am learning Deep Learning" shuffled becomes "Learning I Deep am Learning" — the meaning is destroyed. This is exactly why regular ANNs/CNNs aren't sufficient for this kind of data; we need models that can **preserve and use the order of a sequence**.

Time Series Data

Data collected over **time** — each value is recorded at a specific timestep. Examples: stock prices, weather readings, sensor data, monthly sales figures.

Time series data typically shows patterns like **trend** (rising/falling over time) and **seasonality** (repeating patterns), and the model has to learn what the next value will be based on previous timesteps.

Text Data

Text is also a sequence — words appear in a particular order, and their meaning depends heavily on that order. NLP tasks (translation, sentiment analysis, chatbots) all operate on sequential text data, where each word's meaning is shaped by the words around it.

Speech Data

Speech (audio) is a continuous signal over time, split into small time-frames. Each frame is related to the frames before it — which is why speech recognition and speech-to-text tasks also rely on sequential models.

Video Data

Video is a sequence of images (frames), where each frame is connected to the frame before it (motion, movement). Video processing therefore has to handle both **spatial** structure (CNNs — what's inside each frame) and **temporal** structure (RNN/LSTM — the sequence across frames).

Data Type	Sequence Unit	Example Task
Time Series	Timestamped values	Stock price prediction, weather forecasting
Text	Words / tokens	Translation, sentiment analysis, chatbots
Speech	Audio frames	Speech-to-text, voice assistants
Video	Image frames	Action recognition, video captioning

2. RNN (Recurrent Neural Network)

RNN Concept

An **RNN** is a neural network that processes sequential data step by step, updating a "**memory**" (hidden state) at every step and carrying it forward. This lets an RNN retain context from previous inputs while processing the current input — something a regular ANN cannot do at all, since an ANN treats every input as independent.

RNN Architecture

An RNN's structure is a single "cell" that is reused repeatedly (recurrently) — at every timestep, this same cell takes its input, combines it with its previous memory, and produces a new memory + output.

$$\begin{array}{ccccccc} x_1 \rightarrow [RNN \text{ Cell}] \rightarrow h_1 & \rightarrow [RNN \text{ Cell}] \rightarrow h_2 & \rightarrow [RNN \text{ Cell}] \rightarrow h_3 & \rightarrow \dots & \rightarrow h_T \\ \uparrow & \uparrow & \uparrow & & \\ x_2 & x_3 & x_4 & & \end{array} \quad (\text{same cell/weights reused at every step})$$

Hidden State

The **Hidden State (h)** is the vector that acts as the RNN's "memory." It's updated at every timestep and carries summarized information from all previous inputs — this is what lets the RNN understand the context of a sequence.

Recurrent Connections

In an RNN, the hidden state is fed back into the same layer as an input at the next step — this is called a **recurrent connection**. The current timestep's hidden state becomes part of the input for the next timestep, creating a "loop" in the network (which looks like ordinary stacked layers once "unrolled" across time).

Forward Pass in RNN

At every timestep t , the RNN computes:

$$\begin{array}{ll} h_t = \tanh(x_t \cdot W_{xh} + h_{(t-1)} \cdot W_{hh} + b_h) & \# \text{ new hidden state} \\ y_t = h_t \cdot W_{hy} + b_y & \# \text{ (optional) output at this step} \end{array}$$

Here **W_{xh}** , **W_{hh}** , and **W_{hy}** are weights that are **shared across every timestep** — this weight-sharing is what lets an RNN handle sequences of any length, whether it's a 5-word sentence or a 500-word document.

RNN Limitations

A plain RNN can process a sequence, but it struggles to retain **far-away context** over **long sequences** — the root cause of this is the two gradient problems below.

Vanishing Gradient Problem

When an RNN is trained via **Backpropagation Through Time (BPTT)**, the gradient has to be multiplied repeatedly as it flows backward through time. If these multiplied values are smaller than 1, the gradient shrinks at every step — becoming so small that weights for early timesteps barely update at all. This means the RNN effectively **forgets long-term dependencies**.

Exploding Gradient Problem

The opposite case: if the multiplied values are larger than 1, the gradient grows rapidly and weight updates become so large that training becomes **unstable** (loss can shoot up to NaN or infinity). The common fix is **gradient clipping** — capping the gradient at a maximum value.

```
# Gradient clipping example
np.clip(gradient, -clip_value, clip_value, out=gradient)
```

These two problems (vanishing and exploding gradients) are exactly why LSTM and GRU were designed — to let a network reliably maintain memory across long sequences.

3. LSTM (Long Short-Term Memory)

LSTM Idea

LSTM is a specialized type of RNN designed specifically to solve the vanishing gradient problem of plain RNNs. Its core idea is to maintain a separate "**Cell State**" that acts as long-term memory, controlled by a set of "**gates**" — which decide what to remember, what to forget, and what to output.

LSTM Architecture

Every LSTM cell maintains two states (Cell State and Hidden State) and three gates (Forget, Input, Output) that control these states.

Cell State

The **Cell State (C_t)** acts like a "conveyor belt" that carries information across the entire sequence — it only undergoes minor linear changes (multiplication and addition), so gradients can flow through it without shrinking much. This is exactly how LSTM solves the vanishing gradient problem.

Hidden State

The **Hidden State (h_t)** is the same kind of output/memory as in a plain RNN, passed to the next timestep and next layer — except now it is a filtered version of the Cell State, produced through the Output Gate.

Forget Gate

This gate decides **what information to remove (forget)** from the Cell State:

```
f_t = sigmoid( x_t . Wf + h_(t-1) . Uf + bf )    # outputs 0 (forget) to 1 (keep)
```

Input Gate

This gate decides **what new information to add** to the Cell State:

```
i_t = sigmoid( x_t . Wi + h_(t-1) . Ui + bi )  
C_candidate = tanh( x_t . Wc + h_(t-1) . Uc + bc )    # new candidate information
```

Output Gate

This gate decides **which part** of the Cell State should be exposed as this timestep's hidden state/output:

```
o_t = sigmoid( x_t . Wo + h_(t-1) . Uo + bo )
```

How LSTM Maintains Long-Term Memory

All three gates combine to update the Cell State:

```
C_t = f_t * C_(t-1) + i_t * C_candidate    # keep some old memory, add some new info  
h_t = o_t * tanh(C_t)                      # new hidden state (filtered cell state)
```

Because the Cell State update is mostly based on **addition** (rather than repeated multiplication), gradients flowing through this path shrink far less — this is exactly why LSTM can retain important information even across very long sequences.

RNN vs LSTM

	RNN	LSTM
Memory	Hidden State only	Hidden State + separate Cell State
Gates	None	Forget, Input, Output
Long-range dependencies	Weak (vanishing gradient)	Strong
Parameters	Fewer	More (due to 3 gates)
Training Speed	Faster	Somewhat slower than RNN

4. GRU (Gated Recurrent Unit)

GRU Concept

GRU is a simplified version of LSTM, introduced in 2014 to achieve LSTM-like performance with fewer parameters and less computation. GRU has no separate Cell State — only a single Hidden State — and uses just **two gates** instead of three.

Update Gate

This gate decides **how much of the previous hidden state to carry forward** versus how much new information to add (it effectively combines the role of LSTM's Forget and Input gates):

$$z_t = \text{sigmoid}(x_t \cdot W_z + h_{(t-1)} \cdot U_z + b_z)$$

Reset Gate

This gate decides **how much of the previous hidden state to "forget"** when computing the new candidate hidden state:

$$r_t = \text{sigmoid}(x_t \cdot W_r + h_{(t-1)} \cdot U_r + b_r)$$

GRU Architecture

The Update and Reset gates combine to produce the new hidden state:

$$\begin{aligned} h_{\text{candidate}} &= \tanh(x_t \cdot W_h + (r_t * h_{(t-1)}) \cdot U_h + b_h) \\ h_t &= (1 - z_t) * h_{(t-1)} + z_t * h_{\text{candidate}} \end{aligned}$$

In this formula, the Update Gate (z_t) directly controls how much of the old hidden state to keep and how much of the new candidate to use — a single equation performing the role of both "forget" and "input" together.

GRU vs LSTM

	LSTM	GRU
States	Cell State + Hidden State	Hidden State only
Gates	3 (Forget, Input, Output)	2 (Update, Reset)
Parameters	More	Fewer (~25-30% less)
Training Speed	Slower	Faster
Performance	Better on very long sequences	Comparable to LSTM on short/medium sequences

Practical rule of thumb: if the dataset is small or training speed matters, start with GRU. If the sequence is very long and accuracy is the top priority, try LSTM.

5. Bidirectional RNN

A normal (unidirectional) RNN can only see **past context** when processing timestep t . But in many tasks, **future context** matters just as much — for example, understanding a word in the middle of a sentence often requires knowing the words both before and after it.

Forward RNN

Processes the sequence **from start to end** (left-to-right), capturing past context exactly like a normal RNN:

```
Forward:  h1 -> h2 -> h3 -> ... -> hT
```

Backward RNN

Processes the same sequence **in reverse, from end to start** (right-to-left), so future context can also be captured:

```
Backward: h1 <- h2 <- h3 <- ... <- hT
```

Bidirectional Architecture

Both RNNs (Forward and Backward) run independently on the same input sequence, and at every timestep their hidden states are **concatenated**:

```
output_t = concat( forward_h_t , backward_h_t )
```

This gives the model access to the **full context of the sequence** (both past and future) at every position, which substantially improves prediction quality.

Applications

Application	Why Bidirectional RNN helps
Translation	The correct translation of a word depends on the context of the whole sentence, in both directions
Named Entity Recognition (NER)	Identifying whether a word is a name/place/organization often requires words on both sides
Text Classification	Overall context from both directions of a sentence/paragraph produces better classification

6. Sequence-to-Sequence Models (Seq2Seq)

Many real-world tasks have both input and output as sequences, but of **different lengths** — for example, a 5-word English sentence might translate into a 7-word Urdu sentence. Plain RNN/LSTM can't directly handle this kind of variable-length input-output mapping — which is why the **Seq2Seq (Encoder-Decoder)** architecture was designed.

Encoder-Decoder Architecture

Input Sequence -> [ENCODER: RNN/LSTM/GRU] -> Context Vector -> [DECODER: RNN/LSTM/GRU] -> Output Sequence

Encoder's Role

The **Encoder** processes the input sequence word-by-word (or token-by-token), and compresses the information from the entire sequence into a **fixed-size vector** — usually the encoder's final hidden state.

Decoder's Role

The **Decoder** takes this compressed context and generates the output sequence **one token at a time** — at each step, it uses the previously generated token and its own hidden state as additional input.

Context Vector

The **Context Vector** is the fixed-size representation the encoder builds from the entire input sequence and hands off to the decoder. It's the only connection between the Encoder and Decoder, so it needs to summarize the entire input sequence.

Limitation: for very long sequences, compressing everything into one fixed-size context vector becomes difficult — information can get lost. This exact bottleneck is what later motivated the **Attention Mechanism** (covered in Day 4).

Applications

Application	How Seq2Seq is used
Machine Translation	Encoder encodes the source-language sentence, Decoder generates the target-language sentence
Text Summarization	Encoder reads the full document, Decoder generates a short summary
Chatbots	Encoder understands the user's message, Decoder generates a relevant response

Key Takeaways

- **Sequential Data** (time series, text, speech, video) has order that matters — regular ANN/CNN architectures cannot capture this order.
- **RNN** retains past context through a hidden state, but struggles on long sequences due to the **vanishing/exploding gradient** problem.
- **LSTM** maintains long-term memory effectively via a separate Cell State and three gates (Forget, Input, Output).
- **GRU** is a simplified version of LSTM — two gates (Update, Reset) with fewer parameters and comparable performance.
- **Bidirectional RNN** combines a forward and backward RNN so the model has access to both past and future context.
- **Seq2Seq (Encoder-Decoder)** was designed for variable-length input-output tasks (translation, summarization, chatbots) — the Encoder compresses the input into a context vector, and the Decoder generates output from it.

Module 5 | Day 3 Complete — Next (Day 4): Attention Mechanism, Transformers, and Generative Models (Autoencoders, VAE, GANs).